

Relatório do Projeto Final

Luiz Gustavo Turatti

Fase 3 – Arquitetura de Segurança para o EventMaster

1. Visão Geral da Arquitetura de Segurança

A Fase 3 consolida a transformação do EventMaster em uma plataforma distribuída robusta e de classe empresarial. Para mitigar os riscos listados no *OWASP Top 10* e garantir uma comunicação segura entre microsserviços, a arquitetura incorpora os princípios de *Zero Trust* e o protocolo OAuth 2.0 com autenticação via tokens JWT .

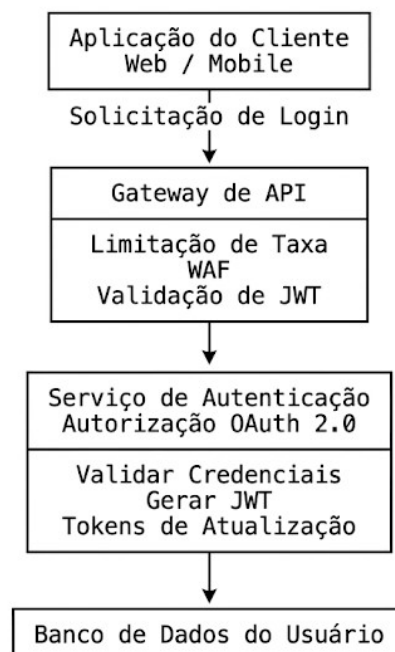
A segurança é tratada como um pilar transversal, integrada nativamente desde a concepção da infraestrutura e das APIs até as regras de autenticação, autorização, proteção de dados e pipelines de implantação (CI/CD).

A topologia proposta baseia-se nos seguintes componentes:

- **Servidor de Autorização OAuth 2.0** para centralizar a gestão de identidades.
- **Autenticação Stateless (JWT)** para garantir escalabilidade.
- **Políticas de Controle de Acesso Baseado em Funções (RBAC).**
- **Ciclo de Vida de Tokens Seguro** e criptografia ponta a ponta entre serviços de back-end.

2. Fluxo de Autenticação Segura (OAuth 2.0 + JWT)

Arquitetura de Login Proposta



Adotou-se o **Authorization Code Flow com PKCE** (Proof Key for Code Exchange). Esta é a abordagem mais segura e recomendada pelo mercado para proteger aplicações Web tradicionais, aplicativos móveis e Aplicações de Página Única (SPAs).

Sequência de Autenticação

1. O usuário submete suas credenciais por meio da interface de login.
2. O **API Gateway** intercepta e encaminha a requisição ao Serviço de Autenticação.
3. O **Serviço de Autenticação** consulta o banco de dados e valida o hash da senha utilizando o algoritmo **BCrypt**.
4. Após a validação com sucesso, o servidor OAuth emite um par de tokens: um **Access Token (JWT)** e um **Refresh Token**.
5. O cliente armazena esses tokens de maneira segura e os anexa ao cabeçalho de cada requisição subsequente.
6. O **API Gateway** valida a assinatura do JWT de forma autônoma e, caso seja legítimo, roteia o tráfego para o microserviço correspondente.

3. Estrutura e Controles de Segurança do JWT

Exemplo de Payload do Token

```
{
  "sub": "12345",
  "email": "user@eventmaster.com",
  "role": "CUSTOMER",
  "iat": 1715900000,
  "exp": 1715903600
}
```

4. Compreendendo o cenário de segurança

4.1. Segurança no API Gateway

O API Gateway atua como a principal linha de defesa e a camada central de segurança da arquitetura. Ele é responsável por interceptar, validar e rotear todas as requisições externas antes que alcancem os serviços internos, garantindo a aplicação uniforme das políticas de segurança.

4.2. Comunicação Segura entre Microserviços (mTLS)

Para a comunicação interna, os microserviços utilizam **mTLS (Mutual TLS)**. Diferente do TLS padrão, o mTLS exige que tanto o cliente (serviço de origem) quanto o servidor (serviço de destino) apresentem certificados digitais válidos.

- **Benefícios:** Garante a autenticação rigorosa entre os serviços, criptografa todo o tráfego interno (evitando interceptação de dados) e impossibilita ataques de falsificação de identidade (*impersonation*).

4.3. Controles de Segurança JWT (JSON Web Token)

O gerenciamento de sessões e autorizações baseia-se em tokens JWT, protegidos pelas seguintes diretrizes:

Controle	Propósito
Tempo de Expiração Curto	Limita a janela de oportunidade para o uso de tokens roubados.
Assinatura de Tokens	Garante a integridade do token, impossibilitando adulterações por parte do cliente.
Refresh Tokens	Permite a reautenticação segura sem expor credenciais repetidamente.
Diretiva <i>HTTPS Only</i>	Impede a interceptação de tokens na rede (ataques <i>Man-in-the-Middle</i>).
Cookies Seguros (HttpOnly)	Mitiga o risco de roubo de tokens via ataques XSS (<i>Cross-Site Scripting</i>).
Rotação de Tokens	Invalida tokens antigos a cada renovação, prevenindo ataques de repetição (<i>Replay Attacks</i>).

5. Mitigação das Vulnerabilidades OWASP Top 10

Abaixo, apresentamos a análise dos riscos associados às dez maiores vulnerabilidades listadas pelo OWASP e os respectivos controles de defesa implementados em nossa solução.

5.1. Quebra de Controle de Acesso (*Broken Access Control*)

- **O Risco:** Ocorre quando os usuários conseguem acessar recursos ou executar ações fora de suas permissões (ex.: um cliente comum acessando *endpoints* exclusivos de administradores).
- **Controles de Defesa:**
 - Implementação de **RBAC (Controle de Acesso Baseado em Funções)**, segregando os privilégios entre perfis como ADMIN, CUSTOMER, ORGANIZER e SUPPORT.
 - **Validação de *Claims* no JWT:** As APIs exigem a verificação interna do token (ex.: role=ADMIN) antes de processar a requisição.
 - **Aplicação no API Gateway:** As rotas sensíveis são protegidas e bloqueadas centralmente.
 - **Princípio do Menor Privilégio:** Cada microserviço possui acesso estritamente limitado aos recursos necessários para sua função.

5.2. Configurações Inseguras (*Security Misconfiguration*)

- **O Risco:** A exploração de senhas padrão, mensagens de erro detalhadas (*stack traces*), portas abertas e repositórios em nuvem mal configurados.
- **Controles de Defesa:**
 - **Containers *Hardened*:** Utilização de imagens mínimas e execução de containers sem privilégios de administrador (*non-root*).

- **Cabeçalhos de Segurança HTTP:** Implementação de políticas como *Content-Security-Policy*, *X-Frame-Options* e *Strict-Transport-Security*.
- **Gerenciamento de Segredos:** Proibição do uso de senhas e chaves fixas no código (*hardcoded*). Utiliza-se gerenciadores de cofres como HashiCorp Vault, AWS Secrets Manager ou Kubernetes Secrets.
- **Desativação do Modo de Depuração:** Ocultação de erros internos e *stack traces* em ambientes de produção.

5.3. Falhas na Cadeia de Suprimentos de Software (*Software Supply Chain Failures*)

- **O Risco:** Comprometimento do sistema através de bibliotecas ou dependências de terceiros maliciosas ou vulneráveis.
- **Controles de Defesa:**
 - **Verificação de Dependências:** Uso contínuo de ferramentas como OWASP Dependency Check, Snyk e Dependabot.
 - **Artefatos Assinados:** Verificação da integridade dos pacotes de *build* antes da implantação.
 - **BOM de Software (SBOM):** Rastreamento rigoroso de cada versão de dependência utilizada no ecossistema.
 - **Registros Confiáveis:** Restrição ao uso de pacotes provenientes apenas de repositórios e registros oficiais validados.

5.4. Falhas Criptográficas (*Cryptographic Failures*)

- **O Risco:** A exposição de dados sensíveis devido à ausência de criptografia ou ao uso de algoritmos fracos.
- **Controles de Defesa:**
 - **Hashes Robustos:** O armazenamento de senhas utiliza algoritmos modernos, resistentes a ataques de força bruta, como BCrypt ou Argon2.
 - **Criptografia em Trânsito:** Toda a comunicação é protegida por TLS 1.3.
 - **Criptografia em Repouso:** Dados sensíveis (como informações de pagamento) são armazenados sob criptografia AES-256.
 - **Rotação de Chaves:** Política periódica de renovação de chaves criptográficas e segredos.

5.5. Injeção (*Injection*)

- **O Risco:** Ocorre quando dados não confiáveis são enviados a um interpretador como parte de um comando ou consulta (como SQL, NoSQL ou Injeção de Comando).

- *Exemplo Vulnerável:* Concatenar *strings* diretamente na consulta: `SELECT * FROM users WHERE email=' + email + '`
- **Controles de Defesa:**
 - **Prepared Statements:** O uso de instruções parametrizadas (ex.: `SELECT * FROM users WHERE email=?`) garante que o banco de dados trate a entrada do usuário estritamente como um dado, e não como código executável.
 - **Frameworks ORM:** Reduzem a necessidade de escrever consultas SQL manuais.
 - **Validação de Entrada:** Bloqueio de *payloads* malformados ou caracteres perigosos.
 - **WAF (Web Application Firewall):** Identificação e bloqueio de assinaturas de ataques conhecidos.

5.6. Design Inseguro (*Insecure Design*)

- **O Risco:** Falhas arquiteturais decorrentes da ausência de considerações de segurança nas fases iniciais do projeto.
- **Controles de Defesa:**
 - **Security by Design:** A segurança é tratada como requisito desde a primeira *sprint*.
 - **Modelagem de Ameaças:** Utilização de metodologias como STRIDE, DREAD e *Attack Trees* antes da fase de implementação.
 - **Arquitetura Zero Trust:** O sistema não confia em nenhuma requisição por padrão; todas devem ser autenticadas e validadas.
 - **Transações SAGA Seguras:** Garantia de integridade de dados e *rollback* automático para prevenir transações ou compras inconsistentes.

5.7. Falhas de Identificação e Autenticação (*Identification and Authentication Failures*)

- **O Risco:** Comprometimento de contas devido a senhas fracas, reutilização de credenciais (*credential stuffing*) ou sequestro de sessão.
- **Controles de Defesa:**
 - **Autenticação Multifator (MFA):** Exigida rigorosamente para perfis de alto privilégio (Administradores e Organizadores).
 - **Política de Senhas Fortes:** Exigência de um mínimo de 12 caracteres, combinando letras maiúsculas, minúsculas, números e caracteres especiais.
 - **Bloqueio de Conta:** Implementação de bloqueio temporário após 5 tentativas de login falhas para mitigar ataques de força bruta.
 - **Controles de Sessão:** Uso de expiração curta para tokens JWT e rotação de *Refresh Tokens*.

5.8. Componentes Vulneráveis e Desatualizados (*Vulnerable and Outdated Components*)

- **O Risco:** O uso de bibliotecas, *frameworks* ou sistemas operacionais legados com vulnerabilidades públicas conhecidas (CVEs).
- **Controles de Defesa:**
 - **Atualizações Automatizadas de Segurança:** Os pipelines de CI/CD possuem etapas automatizadas para barrar *builds* com vulnerabilidades conhecidas.
 - **Containers Imutáveis:** Cada implantação é baseada em uma imagem estática e reproduzível, impedindo alterações no servidor em tempo de execução.
 - **Ciclo de Correções (Patching):** Estabelecimento de um calendário mensal ininterrupto para aplicação de *patches*.

5.9. Falhas de Integridade de Software e Dados (*Software and Data Integrity Failures*)

- **O Risco:** A inserção de código malicioso nos pipelines de integração ou entrega contínua (CI/CD), comprometendo a aplicação antes mesmo de sua publicação.
- **Controles de Defesa:**
 - **Assinatura de Imagens Docker:** Validação criptográfica para assegurar a autenticidade das imagens antes da implantação.
 - **Validação do Pipeline:** Garantia de que apenas pipelines aprovados e confiáveis realizem implantações no ambiente de produção.
 - **Proteção de Ramificações (Git Branch Protection):** Revisão de código (*Code Review*) obrigatória antes de qualquer mesclagem (*merge*) na ramificação principal.
 - **Verificação de Checksums:** Validação rigorosa dos artefatos utilizando o algoritmo SHA-256.

5.10. Falsificação de Requisição do Lado do Servidor (*SSRF — Server-Side Request Forgery*)

- **O Risco:** Uma falha que permite a um invasor induzir o servidor a realizar requisições HTTP para domínios arbitrários, podendo expor serviços internos sensíveis que não estariam acessíveis externamente.
 - *Exemplo Vulnerável:* Aceitar cegamente uma URL fornecida pelo usuário: `URL url = new URL(userInputUrl);`
- **Controles de Defesa:**
 - **Lista de Permissões (URL Allowlist):** O servidor só processará requisições direcionadas a domínios explicitamente confiáveis.
 - **Validação de DNS:** Bloqueio de resoluções para endereços IP de redes internas (ex.: 127.0.0.1, 10.0.0.0/8).
 - **Segmentação de Rede:** Isolamento rigoroso dos serviços internos.

- **Regras de Firewall de Saída (Outbound):** Restrição estrita de quais requisições o servidor está autorizado a realizar para a internet.

6. Tecnologias Recomendadas

Área	Tecnologia
Autenticação	OAuth 2.0
Autorização	JWT
Segurança de Senhas	BCrypt
Gateway	Spring Cloud Gateway
WAF	ModSecurity
Gerenciamento de Segredos	Vault
Monitoramento	Grafana
Análise de Dependências	OWASP Dependency Check
Segurança de Contêineres	Trivy
Provedor de Identidade	Keycloak

7. Arquitetura Segura de Banco de Dados

Segregação por Serviço

- Banco de Dados de Usuários
- Banco de Dados de Catálogo
- Banco de Dados de Pagamentos

Benefício: Limita o raio de impacto (*blast radius*) em caso de incidentes de segurança.

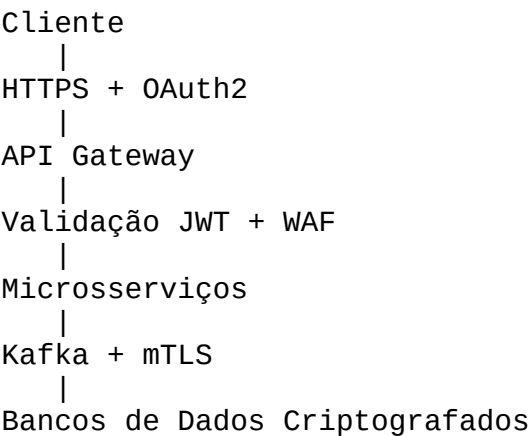
Criptografia em Repouso

- Todos os dados sensíveis são rigorosamente criptografados no banco de dados.

Princípio da Exposição Mínima

- Os serviços de pagamento operam de forma isolada e nunca expõem os dados completos dos cartões de crédito.

8. Arquitetura Segura Final



9. Estratégia de Testes de Segurança

Tipo de Teste	Objetivo Estratégico
Testes de Unidade	Validar a integridade das regras de negócio.
Testes de Integração	Garantir a segurança e o funcionamento das APIs.
Testes de Intrusão (<i>Pentests</i>)	Simular cenários reais de ataques cibernéticos.
SAST	Realizar a análise estática do código-fonte.
DAST	Executar a análise dinâmica da aplicação em tempo de execução.

Tipo de Teste	Objetivo Estratégico
Análise de Dependências	Identificar proativamente bibliotecas e pacotes vulneráveis.

10. Conclusão

A Fase 3 transforma o **EventMaster** em uma plataforma distribuída altamente segura, estritamente alinhada aos modernos padrões de segurança corporativa.

A arquitetura atualizada agora engloba:

- Autenticação via OAuth 2.0;
- Autorização baseada em JWT;
- Controle de acesso baseado em funções (RBAC);
- Mitigação proativa das vulnerabilidades listadas no OWASP Top 10;
- Esteira de CI/CD estruturada com segurança integrada;
- Comunicações ponta a ponta criptografadas;
- Microsserviços blindados;
- Validação em estrita conformidade com o modelo *Zero Trust*;
- Monitoramento de ameaças em tempo real.

A implementação em Java proposta evidencia:

- *Hashing* criptográfico seguro para o armazenamento de senhas;
- Geração robusta de tokens JWT;
- Fluxos de autenticação bem definidos;
- Aplicação de práticas de codificação segura (*Secure Coding*);
- Adoção de padrões de arquitetura defensiva.

Consequentemente, a plataforma resultante consolida-se como:

- Escalável;
- Resiliente;
- Observável;
- Segura desde a concepção (*Secure-by-design*);
- Nativa em nuvem (*Cloud-native*);
- **Pronta para produção** em sistemas de emissão de ingressos de larga escala.